CS6886 Course Project

Madhav (CS23B035) Nitin (EE23B141)

§1 Introduction to the Problem

Spiking neural networks (SNNs) are often called far more energy-efficient than normal networks, because they replace multiplies with additions triggered by sparse spikes. Most SNN accelerators use rate coding, where a value q is sent as q spikes, so a 4-bit activation can cost up to 15 spikes. Time-to-first-spike (TTFS) coding sends a single spike and lets its time carry the value. A quantised network (QNN) with n-bit activations converts exactly into a TTFS network with $T = 2^n - 1$ ticks.

The question is whether this actually saves energy in hardware once memory, control and clocking are counted, and against baselines that are just as optimised. We build a TTFS accelerator tile, TTFS-64, and compare it with SparTen, APEX and Eyeriss using measured power. We then ask where the gain comes from, and which spike code should carry the signed activations of language models.

GitHub repository: Repository Link **Checkpoints:** Drive Link

§2 Prior Work and Baselines

SparTen [1] is a sparse CNN accelerator that multiplies only non-zero weight and activation pairs, using bitmask matching in 32 units. It is our sparse QNN baseline at W4A4. LoAS [2] and APEX [3] are accelerators for rate-coded SNNs; we use APEX with 16 INT4 processing elements at $T = 15$. Eyeriss [4] is the dense reference. Otters [5] turns BERT [10] into a TTFS transformer with binary weights and 4-bit activations, which we use for the language workload. Yan et al. [6] argue on paper that SNN energy savings shrink once memory is counted; we check this in RTL. The CNN is a W4A4 VGG-16 [8] on CIFAR-100 [9] (71.41 % accuracy).

§3 Idea and Implementation

§3.1 Exact TTFS with S/U lanes

A TTFS neuron needs $U = \sum_i w_i\,(T - t_i)$, where t_i is the tick at which input i fired. If spikes arrive in tick order, two registers are enough: on a spike $S \mathrel{+}= w$, and at the end of every tick $U \mathrel{+}= S$. Each weight is then counted exactly $T - t_i$ times, with no multiplier. The neuron fires at the first tick where U crosses a threshold, which gives exactly the QNN output.

§3.2 TTFS-64 architecture

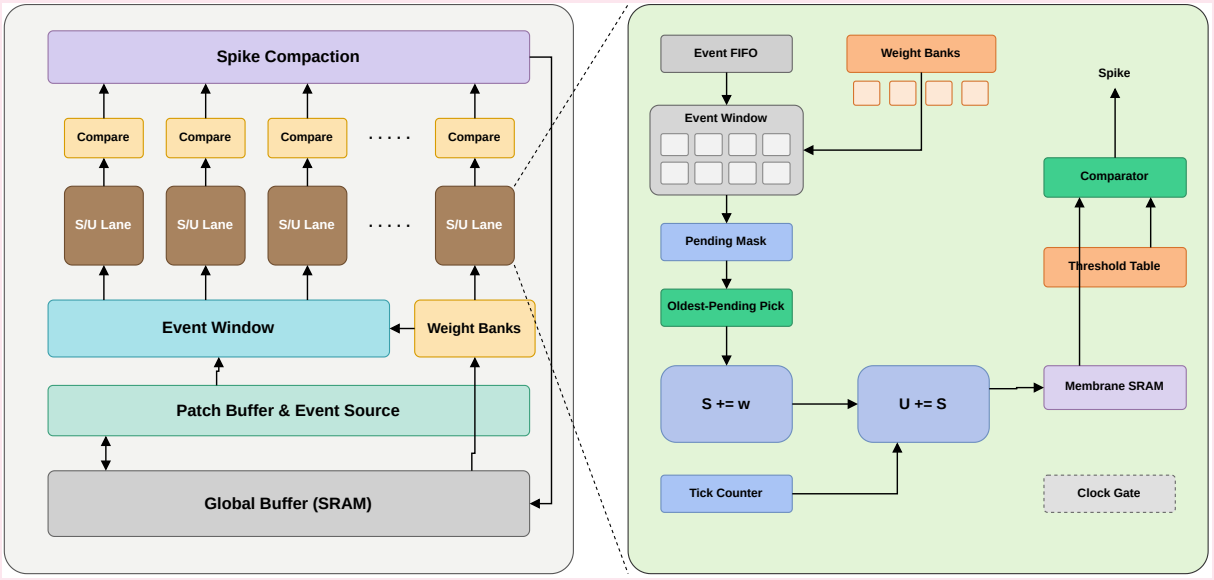


Figure 1: TTFS-64 tile (left) and one S/U lane (right).

The tile computes 64 output channels at a time (Figure 1). Inputs for a group of 32 pixels are loaded once into a patch buffer, and the event source turns them into spikes sorted by tick. Spikes enter a 16-slot event window, and each slot fetches its weight column from four weight banks and sends it to all 64 lanes. Each lane only marks the slots where its own weight is non-zero, and adds the oldest one first, so idle lanes never wait for zero weights.

Inside a lane, S collects weights and U adds S at every tick; ticks with $S = 0$ are skipped. At the end of a pass, U is added into the membrane SRAM. When a group of pixels is done, each lane compares its membrane against a shared threshold table, one tick at a time, and the spikes are compacted and written back. We also added clock gating to each lane, so its registers switch only when something changes. This is exact and cuts lane power by 11–13 %.

§3.3 Polarity TTFS for language models

Transformer activations are signed, but TTFS can only send non-negative values. The common fix adds an offset, which turns the most common value, zero, into a spike. Instead we send $|q|$ as the spike time and the sign as one extra bit, so zero stays silent. Only the magnitude is timed, so we need $T = 8$ ticks instead of 15. On the Otters BERT, all 73 converted layers match the QNN output.

§3.4 Measurement flow

All designs are written in SystemVerilog and simulated in Verilator on all 12 conv layers of one image, with every output checked bit-exactly. Power comes from Design Compiler (NanGate 45 nm, 5 ns) using switching activity from the same simulations. All SRAMs are priced with CACTI 7 [7] for every design alike, and TTFS and SparTen share the same global buffer and weight encoding.

§4 Evaluation

§4.1 CNN results

Table 1: Energy per image (μJ), latency and area; gains relative to TTFS-64.

Sparsity	Design	Energy	Latency (ms)	Area (mm^2)	Energy gain	EDP gain
40 %	TTFS-64 (ours)	**226.2**	**8.40**	**0.334**	–	–
	SparTen W4/A4	396.4	12.15	0.584	1.75×	2.54×
	APEX (T=15)	1421.1	38.37	0.567	6.28×	28.7×
	Eyeriss INT4	2326.8	35.32	1.778	10.3×	43.3×
70 %	TTFS-64 (ours)	**163.7**	**6.22**		–	–
	SparTen W4/A4	270.0	8.28		1.65×	2.19×
	APEX	1078.7	26.10		6.59×	27.6×
98 % (LoAS)	TTFS-64 (ours)	**85.5**	**3.64**		–	–
	SparTen W4/A4	148.6	5.14		1.74×	2.45×
	APEX	646.5	12.16		7.56×	25.3×

TTFS-64 uses 1.65–1.75× less energy than SparTen, is 1.33–1.45× faster and 1.75× smaller. Against the rate-coded APEX the gain is 6.3–7.6×, and Eyeriss, which cannot skip zeros, is 10–25× worse.

§4.2 Where the gain comes from

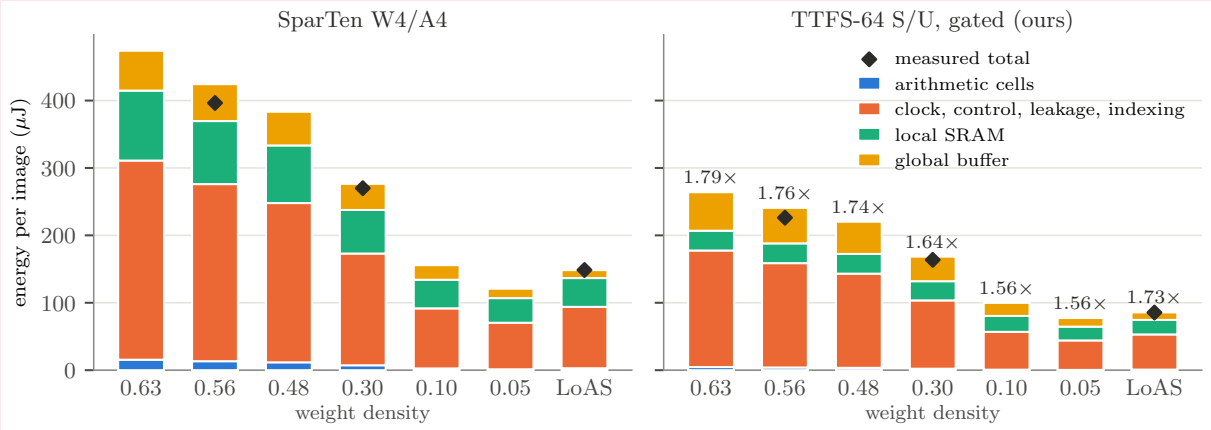


Figure 2: Energy per image by cause across weight densities. Diamonds are measured totals.

Across all weight densities the gain stays between 1.56 and 1.79× (Figure 2). Surprisingly, only 2–5 % of the saving comes from replacing multiplies with adds. Most of it comes from the dataflow: one weight column is shared by 64 small lanes, while SparTen needs matching logic and buffers in each of its 32 units. Arithmetic is only 1–3 % of SparTen's energy, so even a free multiplier would barely help it.

§4.3 Roofline analysis

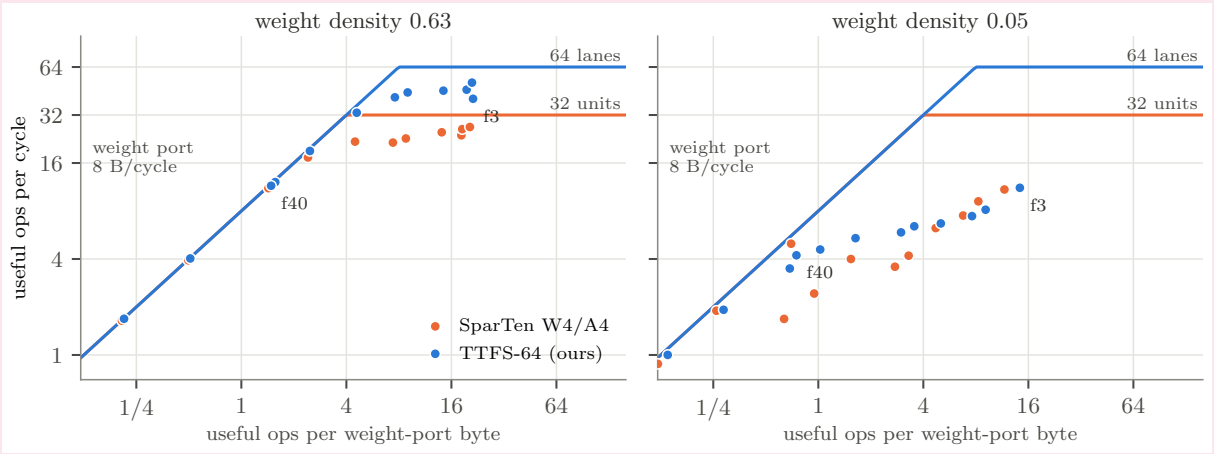


Figure 3: Roofline of the 12 conv layers. Both designs share the same 8 B/cycle weight port.

Figure 3 shows three regimes. Early layers reuse each weight across many pixels and are compute-bound, so TTFS with 64 lanes runs 1.7–1.9× faster than SparTen with 32 units. Late layers have few pixels, sit on the memory roof, and both designs run at the same speed. At very high sparsity every layer falls below both roofs, because control overhead dominates.

§4.4 Language models

Table 2: Spike codes on one BERT-base encoder layer (128 tokens).

Design	W1A4 (μJ)	vs ours	W4A4 (μJ)	vs ours
Polarity TTFS, T=8 (ours)	**1852**	–	**1393**	–
Offset TTFS, T=15	2376	1.28×	1782	1.28×
Bit-serial	2204	1.19×	1654	1.19×
Rate coding	3432	1.85×	2532	1.82×
SparTen W4/A4	3858	2.08×	2746	1.97×
QNN with the same dataflow	1852	1.00×	1474	1.06×

On a real BERT layer, polarity coding is 1.28× cheaper than offset TTFS, since about a quarter of the inputs are zero and now stay silent. It also beats bit-serial and rate coding. However, a QNN with the same dataflow is just as good (1.00–1.06×), which again shows that the gain is in the dataflow.

For LLMs the picture is worse. Decoding reads every weight once per token, so it is memory-bound. For a 7B INT4 model, reading the weights costs about 111 mJ per token while all the multiplies cost about 2 mJ, so spiking can save at most about 2 %.

§4.5 Conclusion

With measured power and fair memory pricing, a TTFS tile is 1.65–1.75× more efficient than a strong sparse QNN accelerator and 6–8× more efficient than a rate-coded SNN accelerator, and polarity coding is the best way to send signed activations. But the gain comes from the dataflow, not from the spike

itself. The biggest wins for spiking hardware are against rate coding. Our main limitations are split synthesis (wires between blocks are not counted) and a single test image.

References

[1] A. Gondimalla, N. Chesnut, M. Thottethodi, T. N. Vijaykumar, "SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks," MICRO 2019.

[2] R. Yin, Y. Kim, D. Wu, P. Panda, "LoAS: Fully Temporal-Parallel Dataflow for Dual-Sparse Spiking Neural Networks," MICRO 2024.

[3] D. Bawa Venkatesh, S. Radhakrishnan, R. Rakshit, G. Srinivasan, "APEX: A Dual-Sparsity Accelerator for Precise and Efficient SNN Inference," arXiv:2608.19046, 2026.

[4] Y.-H. Chen, T. Krishna, J. Emer, V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," IEEE JSSC, 2017.

[5] Z. Yan, J. Mao et al., "Otters: An Energy-Efficient Spiking Transformer via Optical Time-to-First-Spike Encoding," ICLR 2026, https://arxiv.org/abs/2509.18968.

[6] Z. Yan, Z. Bai, K. Tang, W.-F. Wong, "Reconsidering the Energy Efficiency of Spiking Neural Networks Inference from Analytical Perspectives," arXiv:2409.08290, https://arxiv.org/abs/2409.08290.

[7] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, V. Srinivas, "CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories," ACM TACO, 2017.

[8] Y. Chen, Pretrained CIFAR-10/100 models in PyTorch, https://github.com/chenyaofo/pytorch-cifar-models.

[9] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," 2009.

[10] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL 2019.

[11] A. Wang et al., "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding," ICLR 2019.

[12] Project code: https://github.com/Madhav1729/TTFS; checkpoints: https://drive.google.com/file/d/1ULRXTGxmAl8MLo_yqgPObOwExdkZ89UQ/view?usp=sharing.